COSI 107
Assignment 2: Traditional Ciphers and Password Cracking

October 1, 2026

Deadline: 11:59 p.m. on Thursday October 15, 2026

Getting Started

For this assignment, write your solutions in Python 3.10 or later. The required input files are provided in `files.zip`. Task 1 focuses on offline dictionary attacks against password hashes. Python library `hashlib` is required to solve the task. Make sure you familiarize yourself with the library and its functions. Task 2 asks you to break a substitution cipher. In your report, explicitly identify any external resources, tools, or databases you used for Task 2. Read the complete handout before starting.

Task 1: Password Cracking With Dictionary Attacks (60 points)

A dictionary attack is a method to break authentication systems based on the assumption that users are likely to use common words (present in a dictionary) as passwords for accounts. One way to perform such an attack is actively trying to login with multiple password attempts until one guesses the correct password. This however, is infeasible as only a few incorrect attempts. Alternatively, if the attacker somehow gets access to hashes of the users' passwords, they can perform an offline dictionary attack by computing hashes for a large range of dictionary words and then compare them to the actual password hashes. In this case, if two hashes match, the attacker will know the password and can directly authenticate in a single attempt.

Your task in this assignment is to perform an offline dictionary attack to crack passwords of user accounts on a linux server. Linux systems store the details of user accounts and their password hashes in the file: `/etc/shadow`. A simplified version of the `shadow` file from a linux server has been provided in `hw2_files.zip`. While there are many online tools that you can use, in this task you are required to write your own password cracker. To assist you, we have also provided `dictionary.txt` which you can use as a starting point.

The `shadow` file contains a total of 7 users. For this first task, you will crack passwords for `user1` through `user6` (we will come back to `user7` in task 2). Note that password hashes contained in the `shadow` file are of varying levels of difficulty and this will require you to diversify your cracking approach.

- Different passwords have been hashed using different hashing algorithms, `md5, sha1, sha256` etc.
- One password has been hashed along with a SALT.
- A user has enabled a built in additional security feature, in which their password was encoded with a caesar cipher before it was hashed.
- One user has their password in leet speak.

All passwords are present in the *dictionary.txt*, however you need to perform the specific transformation given the security feature applied (e.g., SALT or caesar cipher). You can read more about leet speak here. You may assume a SALT to be of only numeric value, in the range (0-9) and a five-digit decimal string, including leading zeros. The SALT is appended **at the end** of a password before it is hashed.

For example:

- If the password is `helloworld` and the SALT is `12345`, then the hash will be computed as: HASH(helloworld12345).
- If the password is `helloworld` and the SALT is `00001`, then the hash will be computed as: HASH(helloworld00001).

What to Submit?

1. `password_cracker.py`: This is main solution code. Make sure your code compiles and is well commented.
2. `passwords.txt`: Contains all usernames (user[1-7]) and their corresponding passwords that you cracked.

   Autograder format: The passwords will be graded with the autograder and you must submit it with the correct format to receive full credit. Each line must have both username and password separated with a colon:
   `user1:password1`
   `user2:password2`
   ...
   For each user, **submit the original password** as it appears in `dictionary.txt`. Do not submit the salt or any transformed version.
3. `report.pdf`: Section 1: For this task describe in a paragraph or two your approach. Additionally, report the total execution time required for your functions to crack the passwords for users 1-6. You can list the total time taken for each user, specifying the duration in seconds, minutes, or hours. Also mention the approaches you followed to infer the hash algorithm and any other hurdles/difficulties you faced. Explanations must be complete to receive credit.

Task 2: Breaking Substitution Ciphers (40 points)

For this task you will be cracking the password for `user7`. `User7` is smart and decides to create a custom security feature in which the password is encoded using a substitution cipher before it is hashed. `User7` also encrypts all his files and documents in his directory using the same substitution cipher and mapping. The file `encrypted.txt` in `hw2_files.zip` is a piece of encrypted text belonging to `user7` which we have managed to obtain. Your goal in this task is to use the encrypted text to find the mapping of the substitution cipher used by `user7`, and then apply that same mapping to the passwords in `dictionary.txt` to figure out their password by performing a standard dictionary attack.

What to Submit?

1. `analysis.py`: This is the code the performs an analysis on the encrypted text and prints out the mapping, and is well commented.
2. `plaintext.txt`: The plaintext corresponding to the ciphertext in `encrypted.txt`.
3. `report.pdf` Section 2: Describe how you recovered the mapping and plaintext, including any manual steps, code you ran, and online resources you used. Your final code need not reproduce every exploratory step, but your report must explain those steps well enough for us to understand how you reached your solution.

Homework Submission

Submit a single `hw2.zip` containing exactly five files: `password_cracker.py`, `passwords.txt`, `analysis.py`, `plaintext.txt`, and one `report.pdf` with separate sections for Tasks 1 and 2. Reminder: **Submit your work only on Gradescope.**

Good Luck!